

📖 Core Engineering Principles (Chapters 1–4)

- **1. Introduction:** Establishes the core definition of software engineering: engineering discipline focused on all aspects of software production from early stages to system maintenance. It highlights professional/ethical responsibilities and the distinction between generic and customized products.
- **2. Software Processes:** The structured set of activities required to develop a software system. You likely covered the core process activities: **Specification, Design/Implementation, Validation (Testing), and Evolution**. This chapter introduces classic process models like the **Waterfall model, Incremental development, and Integration/Configuration (Reuse-oriented)** frameworks.
- **3. Agile Software Development:** Focuses on rapid software delivery and flexibility to change. Key concepts include the **Agile Manifesto values, Scrum** frameworks (sprints, product backlogs, daily stand-ups), and practices like **User Stories** and continuous integration.
- **4. Requirements Engineering:** The process of establishing what services a customer requires from a system and the constraints under which it operates. This involves elicitation, analysis, documentation (writing functional and non-functional requirements), and validation.

🏗️ Design & Architecture (Chapters 5–7)

- **5. System Modeling:** Developing abstract models of a system to help understand its functionality. This relies heavily on **UML (Unified Modeling Language)** diagrams to show different perspectives:
 - *Context models* (how the system fits into its environment).
 - *Interaction models* (Use Case and Sequence diagrams).
 - *Structural models* (Class diagrams).
 - *Behavioral models* (State machine diagrams).
- **6. Architectural Design:** The critical link between design and requirements engineering. It focuses on how a software system should be organized and how its overall structure is designed. You likely studied architectural patterns like **Model-View-Controller (MVC), Layered architecture, Repository architecture, and Client-Server** models.
- **7. Design and Implementation:** Translating an architectural design into an executable system. This covers object-oriented design processes, identifying design patterns (like the *Observer* or *Strategy* pattern), and open-source development practices.

🔍 Verification & Maintenance (Chapters 8–9)

- **8. Software Testing:** Validating that the software meets its requirements and discovering bugs before deployment. Key testing methodologies include:
 - **Development testing** (Unit testing, Component testing, and System testing).
 - **Test-Driven Development (TDD)** (writing tests *before* writing code).
 - **Release testing** (Black-box testing to satisfy requirements).
 - **User testing** (Alpha, Beta, and Acceptance testing).
- **9. Software Evolution:** Acknowledges that software is dynamic and must change over its lifetime to remain useful. This covers the evolution process, legacy system management, and Lehman's laws of software evolution (e.g., the law of continuing change).

Since you are diving into **Project Planning** next, having **Agile Development (Ch. 3)** fresh in your mind will make **Agile Planning (23.4)** a breeze, and your past knowledge of **Software Testing (Ch. 8)** connects directly to how you allocate time for verification in **Project Scheduling (23.3)**.

Detailed Breakdown of Your Focus Areas

🔧 Chapter 22: Project Management

Software project management is data-driven and focus-heavy because software is intangible, and projects are often uniquely tailored. You need to master three core branches here:

- **22.1 Risk Management:** Identifying, analyzing, and mitigating potential hazards to the project's timeline, budget, or quality. You will look at project risks (affecting schedule/resources), product risks (affecting quality/performance), and business risks (affecting the organization).
- **22.2 Managing People:** Understanding the human element—motivation (Maslow's hierarchy of needs), cohesion, and alignment of engineering staff.
- **22.3 Teamwork:** Choosing team structures (e.g., authoritarian vs. democratic vs. agile matrix teams) and managing communication boundaries to prevent productivity bottlenecks.

📅 Chapter 23: Project Planning

This section is highly analytical and breaks down how a project is structurally mapped from inception to delivery:

- **23.1 Software Pricing:** Understanding how organizations calculate the cost of software. It's not just engineering hours; it factors in organizational, economic, and market pressures.
- **23.2 Plan-Driven Development:** An approach where the project plan is created up front, and managers use it to track progress against distinct milestones.
- **23.3 Project Scheduling:** Splitting software work into discrete tasks, estimating time/resources, and mapping dependencies using tools like **PERT/CPM charts** and **Gantt charts** to discover the *critical path*.
- **23.4 Agile Planning:** Contrast to plan-driven; an iterative approach featuring sprint planning, user story point estimations, and adaptive features based on continuous feedback.
- **23.5 Estimation Techniques:** Moving past guesswork. You'll study algorithmic models and expert judgment approaches (like the Delphi technique) to estimate development effort.
- **23.6 COCOMO Cost Modeling:** The **COnductive COSt Model**—a classic, highly structured algorithmic cost estimation model that uses formulas based on project size (typically in lines of code or function points) and specific cost drivers to predict effort and schedule.